

HXC-159 05_catalogue_survey

HXC-159 — §11.4.74 Catalogue Survey: reuse before rewrite

Table of contents

1. CLI authentication verification
 2. Survey scope and method
 3. Primary target — what HelixSkills actually is
 4. The three skills corpora
 5. Catalogue-Check verdicts
 - Verdict 1 — the skills system itself
 - Verdict 2 — spec-driven development / SpecKit
 - Verdict 3 — upstream SpecKit itself
 - Verdict 4 — third-party skills marketplace / package manager
 - Verdict 5 — anti-bluff mechanical seams
 - Verdict 6 — HelixAgent-side skills corpus
 - Verdict 7 — the SpecKit ↔ Superpowers Bridge extension
 - Verdict 8 — constitution-derived extensions in scope
 - Verdict 9 — a Go loader for Anthropic-format skills
 6. Dependency graph and required layout
 7. Honest boundary
- Sources verified 2026-07-29

HXC-159 — §11.4.74 Catalogue Survey: reuse before rewrite

Field	Value
Document	05_catalogue_survey.md
Work item	HXC-159 (Task, In-progress, High)
Phase	1b — advance research + risk (parallel to inventory, §11.4.150(F))
Revision	1
	2026-07-29

Field	Value
Last modified	
Author	(T1/main - claude1 - opus - xhigh)
Status summary	Complete. 2 orgs × 2 forges surveyed with verified CLI auth. 4 reuse verdicts, 2 extend verdicts, 1 no-match.
Anchors	\$11.4.74 (catalogue-first), \$11.4.28(C)/CONST-051(C) (dependency layout), \$11.4.6 (no-guessing), \$11.4.8 (cite or declare original)

Table of contents

1. [CLI authentication verification](#)
2. [Survey scope and method](#)
3. [Primary target — what HelixSkills actually is](#)
4. [The three skills corpora](#)
5. [Catalogue-Check verdicts](#)
6. [Dependency graph and required layout](#)
7. [Honest boundary](#)

1. CLI authentication verification

\$11.4.74 requires surveying both forges; \$11.4.6 requires proving the survey was authenticated, because **an unauthenticated empty list is a false negative, not a finding**. Both CLIs were verified before any result was trusted.

```
$ gh auth status
github.com
  ✓ Logged in to github.com account milos85vasic
  - Active account: true
  - Token scopes: 'admin:public_key', 'gist', 'read:org', 'repo'
```

```
$ glab auth status
gitlab.com
  ✓ Logged in to gitlab.com as milos85vasic (GITLAB_TOKEN)
  ✓ REST API Endpoint: https://gitlab.com/api/v4/
```

Both authenticated. The `read:org` scope on the GitHub token is what makes the private-repo rows in the org listing trustworthy rather than silently truncated.

Caveat carried honestly (§11.4.6): `glab api "groups/HelixDevelopment/projects"` returned a single element that did not parse as a project list. This is an *unresolved* query-shape problem on my side, not proof the group is empty. The GitLab HelixDevelopment enumeration is therefore recorded as **NOT VERIFIED**, and no conclusion is drawn from it. GitLab `vasic-digital` enumerated cleanly (100 projects) and IS verified.

2. Survey scope and method

Forge	Org	Method	Result
GitHub	HelixDevelopment	<code>gh repo list --limit 200 --json</code>	41 repos — VERIFIED
GitHub	vasic-digital	<code>gh repo list --limit 300 --json</code>	300-cap listing — VERIFIED
GitLab	vasic-digital	<code>glab api groups/.../projects</code>	100 projects — VERIFIED
GitLab	HelixDevelopment	<code>glab api groups/.../projects</code>	NOT VERIFIED (query-shape error, see §1)

Search keys: skill, spec, plugin, extension, plus manual read of every HelixDevelopment description.

3. Primary target — what HelixSkills actually is

This is the load-bearing finding of the whole survey, and it contradicts the premise of the task as stated.

Repo: `HelixDevelopment/skills @315b56ce1e4c2570610e0b214ea34d64fc469e1e` (branch `main`, pushed 2026-07-18T02:47:34Z, 8.2 MB, GitHub-detected language: **Shell**).

Measured from the full recursive git tree (?recursive=1, truncated: false — so this is the complete tree, not a page):

Measurement	Value	Consequence
Total tree entries	883 (657 blobs)	—
.go files	237	but see next row
.go files outside docs/	0	there is no shipped Go package
go.mod files	1, at docs/research/mvp/Agent_AI_Skill_Tree_Development/project/go.mod	the only module is inside a <i>research</i> path
SKILL.md files	0	ships no skill in the Anthropic skill format
helix-deps.yaml at root	absent (only inside the research MVP)	CONST-054 manifest missing
tests/	4 shell scripts, all *constitution_inheritance*	no test of the skills system itself
Submodules	exactly 1 — constitution	—
Blob distribution	docs/research 416, docs/repos 185, docs/skills 28, docs/catalog 5, docs/guides 1	~96 % of content is documentation

What its own README says it installs (README Revision 3, 2026-07-18):

Constitution Skills (7 installed via `register.sh`)

action-prefix-system · media-validator · multitrack · reporting-workable-items · scheduled-work-queue · session-sync · workable-item-lifecycle — all at `constitution/skills/<name>/`

Therefore: HelixSkills is not a skills *library*. It is a **registry/catalogue and installer over the constitution submodule's own skills/directory**, plus a large research corpus and an unshipped Go MVP.

And HelixCode already vendors that exact payload. Verified locally:

```
$ ls constitution/skills/
action-prefix-system  media-validator  multitrack  reporting-
workable-items
scheduled-work-queue  session-sync     workable-item-lifecycle
```

Seven directories, matching the README's seven **exactly**. The thing the mandate asks to "incorporate" is, in its shipped payload, already present in HelixCode.

4. The three skills corpora

Surveying for reuse surfaced that "skills" in this ecosystem is not one thing. There are **three disjoint corpora**, and no component reconciles them.

#	Corpus	Location	SKILL.md count	Owner
1	Constitution skills	constitution/skills/ (in HelixCode and in HelixSkills)	3 (of 7 dirs)	HelixDevelopment/constitution
2	HelixAgent skills	submodules/helix_agent/skills/ — plain tracked tree 78ebdc67, 15 categories	1174	HelixDevelopment/agent
3	"HelixSkills System"	HelixDevelopment/skills	0	HelixDevelopment/skills

Name overlap between corpus 1 and corpus 2: **zero** (verified by comm -12 on the sorted directory listings).

Two consequences follow directly, both provable:

(a) The mandate's premise is inverted. The corpus with 1174 skills (HelixAgent's) is the one *not* named in the task; the one named in the task ships zero skills. Incorporating (3) into HelixCode + HelixAgent adds a catalogue whose catalogued payload HelixCode already has, while leaving the 1174-skill corpus unreconciled.

(b) Corpus 1 is internally inconsistent — filename case is split. On a case-sensitive Linux filesystem this is a live loader defect, detected in advance exactly as the operator's mandate requires:

```
HAS SKILL.md : action-prefix-system
NO  SKILL.md : media-validator          -> ships skill.md
(lowercase)
NO  SKILL.md : multitrack               -> ships register.sh ONLY –
no skill doc at all
HAS SKILL.md : reporting-workable-items
NO  SKILL.md : scheduled-work-queue     -> ships skill.md
(lowercase)
NO  SKILL.md : session-sync             -> ships skill.md
(lowercase)
HAS SKILL.md : workable-item-lifecycle
```

A loader globbing SKILL.md finds 3 of 7. A loader globbing skill.md finds 3 *different* ones. multitrack is found by **neither** — it has no skill document in either case. Any incorporation that does not first normalise this will silently load fewer than half the skills it claims to.

5. Catalogue-Check verdicts

Per §11.4.74, one verdict per capability the task would otherwise re-implement.

Verdict 1 — the skills system itself

Catalogue-Check: extend HelixDevelopment/
skills@315b56ce1e4c2570610e0b214ea34d64fc469e1e

Reuse-in-place is not available: the repo has no consumable artifact to reuse (0 SKILL.md, 0 shipped .go, no root go.mod, no root helix-deps.yaml). Per §11.4.74's extend-don't-reimplement rule the missing capability must be built **into that repo** and consumed from there — never re-implemented as a HelixCode helper. The extension work is:

promote the research MVP out of docs/research/... into a real module, add a root helix-deps.yaml (CONST-054), and make it the component that reconciles corpora 1 and 2.

Verdict 2 — spec-driven development / SpecKit

Catalogue-Check: reuse HelixDevelopment/
specifier@55d9eaf14b1fa769574d98c33fd19f2d0948468c

This is the survey's highest-value hit. The task mandates "GitHub SpecKit for all phases, bridged to Superpowers". An own-org repo *already does exactly that*:

HelixDevelopment/specifier — "Spec-Driven Development Fusion Engine — Unified specification intelligence fusing SpecKit + Superpowers + GSD for HelixAgent AI debate ensemble" Go, 1.6 MB, main @ 55d9eaf1 (2026-06-22). Root tree already carries go.mod, go.sum, **helix-deps.yaml**, pkg/, challenges/, tests/, scripts/, upstreams/, CONSTITUTION.md.

It is structurally *more* mature than skills (real module, CONST-054 manifest, challenges). Standing up a fresh SpecKit integration for HXC-159 without consuming specifier would be a \$11.4.74 duplicate-implementation violation. Note it targets "HelixAgent" — one of HXC-159's two consumers — so the bridge partly exists.

Verdict 3 — upstream SpecKit itself

Catalogue-Check: reuse vasic-digital/caf-spec-kit

An org-controlled mirror of github/spec-kit already exists, so the upstream toolkit is vendored and pinnable without adding a new third-party remote.

Verdict 4 — third-party skills marketplace / package manager

Catalogue-Check: reuse vasic-digital/caf-bridle

"425 plugins, 2,810 skills, 200 agents for Claude Code. Open-source marketplace at tonsofskills.com with the ccpi CLI package manager."

Directly relevant prior art for distribution/versioning of a skills corpus, and already in the catalogue. Also vasic-digital/caf-codex-skills ("Skills Catalog for Codex") and vasic-digital/caf-ui-ux-pro-max (an AI *skill*) as secondary references.

Verdict 5 — anti-bluff mechanical seams

Catalogue-Check: reuse vasic-digital/anti_bluff

"Mechanical anti-bluff seams for Helix-constitution projects: needed measurements (SOL-03), DB-layer status custody (SOL-01), evidence-class-at-closure (SOL-04)."

The §11.4.226 evidence-class machinery HXC-159's gates will need already exists.

Verdict 6 — HelixAgent-side skills corpus

Catalogue-Check: extend HelixDevelopment/agent@<pin> (tree 78ebdc67 for skills/)

The 1174-skill corpus lives inside HelixAgent as a plain tracked tree, not a reusable submodule. Any cross-consumer parity work must extend it there or lift it out — it cannot be reused as-is by HelixCode.

Verdict 7 — the Speckit ↔ Superpowers Bridge extension

Catalogue-Check: reuse HelixDevelopment/constitution@ce3331a1ca3793ae52a9f7690fd5c1f8c3e4cbbb

Added per the mid-flight operator requirement that Speckit and Superpowers be used **with the Bridge extension and all constitution-derived extensions**. The Bridge is own-org, already designed, and inherited by reference (§11.4.228) — so HelixSkills must compose **with** it, not beside it.

Measured, 2026-07-29. constitution/docs/research/extensions/speckit_superpowers/implementation/ holds **8 Markdown documents**, each with .html/.pdf/.docx siblings (§11.4.65): README, IMPLEMENTATION_PLAN, NANO_TASK_ENGINE, EXTENSION_DEVELOPMENT (92 KB), TDD_INTEGRATION, CONSTITUTION_INTEGRATION (50 KB), SECURITY, APPENDIX (79 KB) — plus request_with_materials.md one level up. A real, substantial design corpus, not a stub.

But reuse is qualified, and the qualification is the point: of its 7 layers, **L3 SuperSpec, L4 SuperB and L5 SuperBridge MCP are NOT installed on this host** (probed directly — see 06_risk_register.md R-20). L2 Spec-Kit Core is real (specify 0.13.5.dev0), and L6/L7 (Helix LLM, llama.cpp) are present. So the *design* is reusable today; the *execution pipeline* is not yet runnable.

And two of its layers are not own-org. SuperSpec (github.com/WangX0111/superspec) and SuperB (speckit-community.github.io/extensions/superb) fall outside vasic-digital / HelixDevelopment, so they are a §11.4.74 no-match → vendor path with the supply-chain exposure that implies — tracked as R-21. Their existence, maintenance and installability were **not established** at the time of writing; only their absence from this host's PATH is proven (§11.4.6).

Verdict 8 — constitution-derived extensions in scope

Catalogue-Check: reuse HelixDevelopment/constitution@ce3331a1 (all classes below)

Inherited by reference, never copied (§11.4.228 / §11.4.177):

Class	Count	Members
constitution/ skills/	7	action-prefix-system, media-validator, multitrack, reporting-workable-items, scheduled-work-queue, session-sync, workable-item-lifecycle
constitution/ mcp/	2	media-validator-mcp.json, scheduled-work-mcp.json
constitution/ plugins/	2	helix, scheduled-work
constitution/ actions/	—	registry.yaml (§11.4.140), subagent_tiering.yaml
constitution/ scripts/hooks/	7+	live guards — one blocked this session's first agent dispatch, so they are demonstrably live, not decorative

Gap found: §11.4.164's post_update_hook.sh exists in the constitution (20,975 bytes, executable) but the consumer-side registrars it invokes — scripts/register_skills.sh, scripts/register_mcp.sh — **do not exist in HelixCode**. Auto-propagation is half-wired. Tracked as R-24.

Verdict 9 — a Go loader for Anthropic-format skills

Catalogue-Check: no-match 2026-07-29

No repo in either org, on either forge, provides a Go package that discovers, validates, and loads Anthropic-format SKILL.md skills. helix_code/internal/mcp and helix_code/internal/hooks exist as adjacent extension seams but neither is a skill loader. **This is the one genuinely new component** — and per §11.4.74 it belongs *in* HelixDevelopment/skills (Verdict 1), not in HelixCode.

6. Dependency graph and required layout

Measured, not assumed:

```
HelixCode (this repo)
├─ constitution/                @ ce3331a1    (= upstream tip
2026-07-28)
├─ submodules/helix_agent/      → dev.helix.agent, go 1.26, 59
submodules
└─ skills/                     (plain tree 78ebdc67,
1174 SKILL.md)
```

```
HelixDevelopment/skills @ 315b56ce
├─ constitution/              @ 68875c7a    ← 46 commits BEHIND
HelixCode's pin
```

§11.4.28(C) / CONST-051(C) violation if added naively. Adding skills as a submodule of HelixCode would place constitution at two different paths, pinned independently — a nested own-org submodule chain, which is explicitly forbidden:

Nested own-org submodule chains are FORBIDDEN. Add the dependency at the root; the consuming submodule reaches it via documented import/SDK/runtime resolver.

The skew is already real and **already 46 commits wide**:

```
$ git -C constitution log --oneline 68875c7a..ce3331a1 | wc -l
46
skills pin date:      2026-07-18 03:42:37 +0500
helixcode pin date:   2026-07-28 15:19:07 +0500
```

And the skew is **not cosmetic** — it changes the skills payload itself:

```
$ git -C constitution diff --stat 68875c7a..ce3331a1 -- skills/
skills/reporting-workable-items/SKILL.docx | Bin 14458 -> 14458
bytes
skills/reporting-workable-items/SKILL.pdf | Bin 80490 -> 80489
bytes
skills/workable-item-lifecycle/SKILL.docx | Bin 0 -> 14102 bytes
skills/workable-item-lifecycle/SKILL.html | 496 ++++++
+++++
skills/workable-item-lifecycle/SKILL.pdf | Bin 0 -> 79245 bytes
5 files changed, 496 insertions(+)
```

workable-item-lifecycle's rendered skill documents **do not exist** at the pin HelixSkills carries. A consumer resolving skills through HelixSkills' pin gets a demonstrably different — and smaller — skill set than one resolving through HelixCode's. This is version skew with observable capability loss, today, before any integration work has begun.

Required layout (CONST-051(C)): `skills` lands at the consumer root as a sibling of `constitution`, its own nested constitution chain removed, and both consumers resolve the single root-level `constitution`.

7. Honest boundary

Per §11.4.6, this survey establishes what the catalogue *contains* and what the repositories *measurably are*. It does **not** establish that any surveyed repo is correct, tested, or fit for purpose — specifier in particular is asserted here only on its README, description, and root tree; its actual capability is unverified and must be probed before Verdict 2 is acted on.

The GitLab HelixDevelopment enumeration is **NOT VERIFIED** (§1). If a skills-related repo exists only in that group, this survey would have missed it. Re-running with a corrected query is a tracked follow-up, not a closed item.

Sources verified 2026-07-29

All rows below were produced by authenticated CLI calls during this session.

- `gh auth status, glab auth status` — auth verification, accessed 2026-07-29
- `gh repo list HelixDevelopment --limit 200 --json name,description,updatedAt,isPrivate` — accessed 2026-07-29
- `gh repo list vasic-digital --limit 300 --json name,description,updatedAt` — accessed 2026-07-29
- `gh api repos/HelixDevelopment/skills + .../git/trees/main?recursive=1 + .../branches + .../contents/README.md + .../contents/.gitmodules` — accessed 2026-07-29
- `gh api repos/HelixDevelopment/specifier + .../git/trees/main` — accessed 2026-07-29
- `gh api repos/HelixDevelopment/constitution/commits/main` — accessed 2026-07-29
- `glab api "groups/vasic-digital/projects?per_page=100&include_subgroups=true"` — accessed 2026-07-29
- Local worktree measurements: `git ls-tree HEAD constitution, git -C constitution log 68875c7a..ce3331a1, git -C constitution diff --stat ... -- skills/, find ... -name SKILL.md, comm -12` — accessed 2026-07-29